Saúde Memora

Como o sistema funciona, de ponta a ponta

Documentos médicos digitalizados com OCR e IA,
organizados numa linha do tempo e consultáveis por chat,
com idempotência, isolamento por paciente e foco em LGPD.

Aplicação: https://saude-memora.vercel.app

Código: https://github.com/MatheusRafaDev/saude-memora

Documento elaborado a partir do código-fonte e da documentação do repositório.

1. Visão geral

O **Saúde Memora** resolve um problema comum: receitas, exames, laudos e atestados espalhados em papéis e fotos, que desaparecem justamente quando o médico pede. O usuário fotografa ou envia o documento; o sistema **lê, estrutura e cataloga** os dados numa linha do tempo e permite **conversar com o próprio histórico** por um chat que responde citando os documentos originais.

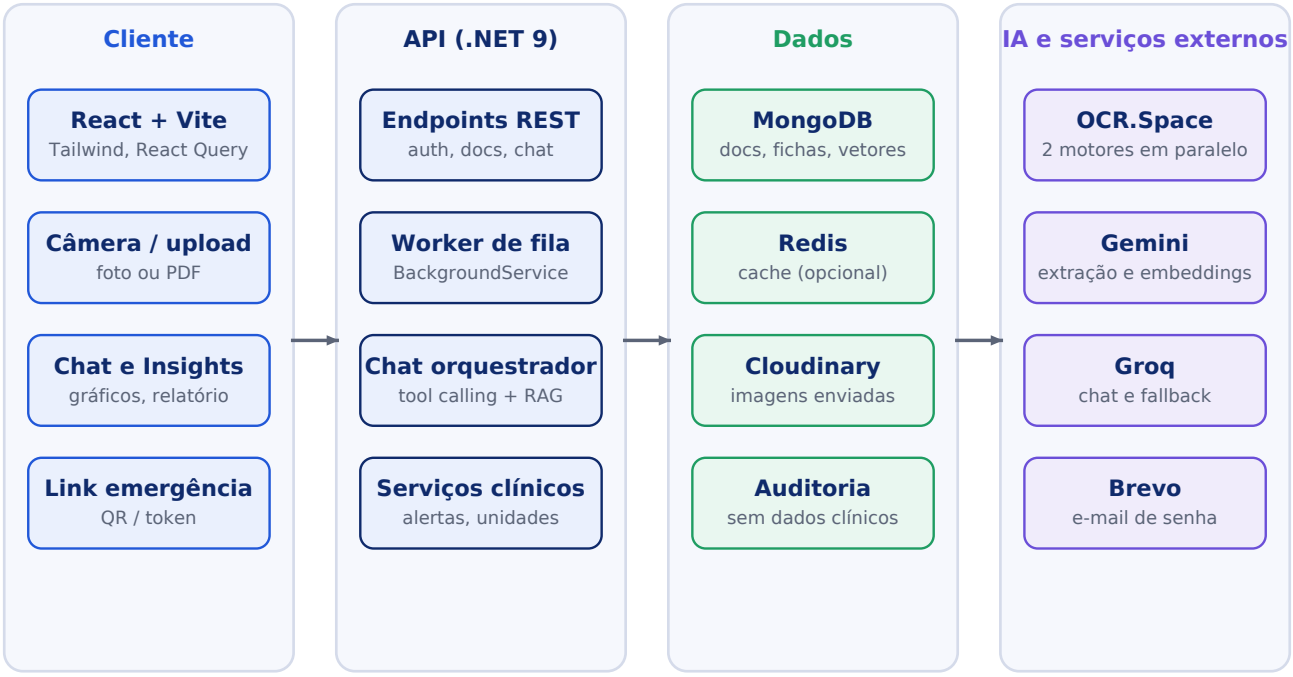
O que o sistema entrega

Módulo	O que faz
Upload inteligente	Foto pela câmera ou arquivo (JPG, PNG, PDF até 10 MB), com seleção do tipo: exame, receita, laudo, atestado, vacina, encaminhamento.
Processamento por IA	OCR duplo, extração estruturada por tipo de documento, normalização de unidades, validação de CID-10/CNES e alertas clínicos.
Revisão assistida	Campos com baixa confiança ficam destacados; o usuário edita e confirma antes de concluir a revisão.
Dashboard e arquivo	Contagens, medicamentos mais frequentes, filtros por tipo/status/período e busca por título, médico, clínica e conteúdo.
Chat com RAG	Pergunta em linguagem natural; a resposta vem dos documentos do próprio paciente, com fontes clicáveis.
Insights	Relatório consolidado, série histórica de exames com faixas de referência e detecção de valores atípicos, e consulta a catálogos.
Ficha médica	Anamnese: alergias, doenças crônicas, medicamentos contínuos, cirurgias, tipo sanguíneo, histórico familiar.
Link de emergência	Token temporário (QR/URL) que expõe só os dados essenciais para socorro, com expiração e revogação.
Privacidade (LGPD)	Consentimento separado para OCR/IA e para chat, auditoria de acesso e exclusão de conta em cascata.

> **Uso responsável**
> O sistema organiza informações; não é diagnóstico nem aconselhamento médico. Dados extraídos por OCR e IA podem conter erros e devem ser conferidos com o documento original e com um profissional de saúde.

2. Arquitetura

O backend é uma **Minimal API em ASP.NET Core 9** organizada em camadas (Api, Application, Domain, Infrastructure). O frontend é um app **React + Vite**. Os dois são publicados juntos na Vercel: o frontend como site estático e a API como serviço em container, com reescrita de `/api/*` para o serviço da API.



Camadas do backend

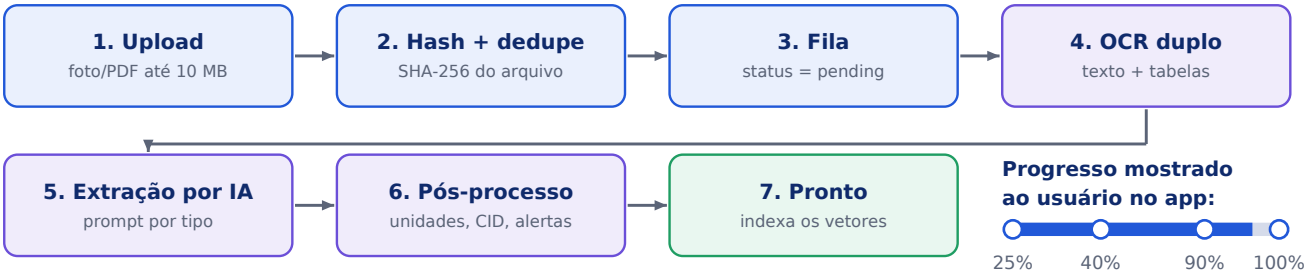
Camada	Responsabilidade	Exemplos
Api	Endpoints, autenticação, rate limiting, CSRF, CORS, worker de fila, orquestração do chat.	Program.cs, ChatEndpoints, DocumentProcessingWorker, PatientChatOrchestrator
Application	Contratos (interfaces), DTOs e exceções.	IAlertaMedicamentoService, IFichaMedicaRepository
Domain	Entidades e interfaces de repositório.	RegistroDocumento, Paciente, FichaMedica, DocumentChunk, AuditoriaAcesso
Infrastructure	Acesso a dados, integrações com IA e serviços clínicos.	DocumentRepository, GeminiEmbeddingService, UnitNormalizer, ResilientHttpSender

Dados no MongoDB

- **Documentos** (RegistroDocumento): imagens, tipo, status, progresso, campos extraídos, medicamentos, resultados de exame, alertas, confiança, hash do arquivo, dados de lock da fila.
- **Pacientes e fichas médicas**, **histórico de chat**, **chunks vetoriais** (DocumentoChunks), **auditoria de acesso**, **logs do sistema**.
- **Catálogos de referência**: CID-10, CNES (estabelecimentos) e medicamentos (bulário ANVISA), importados por endpoints/scripts.
- **Migrações versionadas e idempotentes**, executadas antes de a aplicação aceitar tráfego.

3. Ciclo de vida de um documento

O upload é rápido e o processamento pesado é **assíncrono**: a API responde 202 Accepted assim que o arquivo está salvo, e um worker em segundo plano (BackgroundService) conclui o trabalho enquanto o app mostra o progresso.



Fila e concorrência

- O worker pega o próximo documento com **FindOneAndUpdate** atômico: muda o status para *processing*, grava quem pegou, define um **lock de 3 minutos** e incrementa a versão. Dois workers nunca recebem o mesmo documento.
- Documentos com lock expirado voltam para *pending* (ResetExpiredProcessing), então um worker que caiu no meio não deixa documento preso.
- Concorrência configurável por instância (`DOCUMENT_WORKER_CONCURRENCY`, de 1 a 16; padrão 3). Tentativas futuras respeitam `NextAttemptAt`.
- O processamento só ocorre se o paciente tiver **consentimento ativo para OCR/IA**.

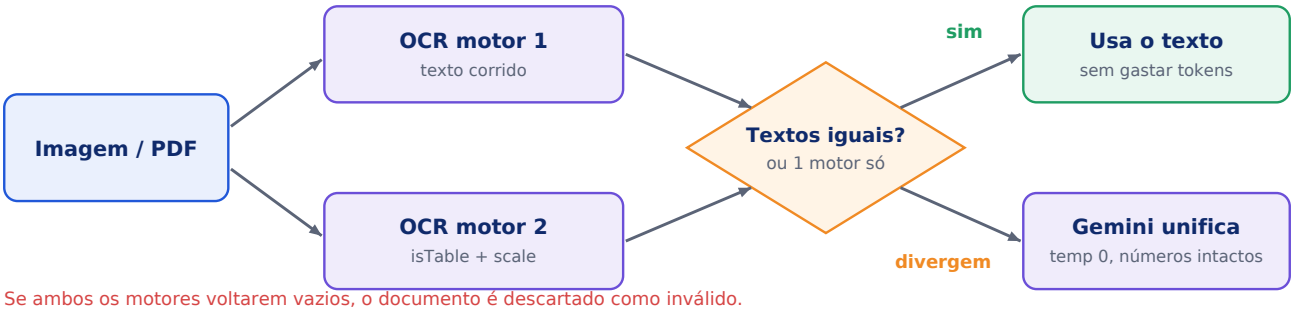
Outras operações sobre o documento

Endpoint	Efeito
POST /api/documents/upload	Valida, deduplica por hash, salva no Cloudinary e no Mongo (status pending).
POST /api/documents/{id}/retry	Recoloca na fila um documento que falhou.
POST /api/documents/{id}/reprocessar	Reprocessa o documento e refaz a indexação vetorial.
PUT /api/documents/{id}	Edição manual dos campos extraídos.
POST .../revisao/confirmar	Conclui a revisão (só se não houver campos de baixa confiança pendentes) e regenera os alertas.
POST .../alertas/{id}/dispensar	O usuário dispensa um alerta.
DELETE /api/documents/{id}	Exclui o documento e seus vetores.

4. IA: leitura e extração

4.1 OCR duplo

Documentos médicos brasileiros misturam receitas e atestados corridos com laudos cheios de tabelas. Por isso o sistema chama o OCR.Space **duas vezes em paralelo** (português, timeout de 45 s por motor): o motor 1 para texto corrido e o motor 2 com `isTable=true` e `scale=true`, que evita embaralhar colunas em exames de sangue.



Se ambos os motores voltarem vazios, o documento é descartado como inválido.

4.2 Unificação sem alterar números

Quando os dois textos divergem, o Gemini os unifica com **temperatura 0**. A instrução é categórica: corrigir quebras ortográficas causadas pelo scanner, mas **nunca alterar dosagens, números, casas decimais ou unidades**. Se os textos são iguais ou só um motor respondeu, nenhum token de IA é gasto.

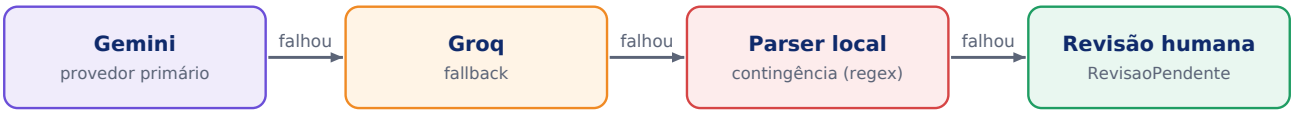
4.3 Prompts por tipo e saída estruturada

Cada tipo de documento tem um prompt versionado (`medical-document-extraction-v2`) e a resposta é JSON estrito:

Tipo	Foco da extração
Receita	Medicamentos, dosagem, frequência, via, CRM do prescritor e CNES.
Atestado	Período de afastamento, data de início, CID-10 citado.
Laudo / exame de imagem	Impressão diagnóstica; achados, técnica e conclusão.
Exame laboratorial	Lista estruturada de resultados com valor, unidade e faixa de referência.
Vacina	Lote, fabricante, número da dose, próxima dose.
Carteirinha	Operadora e número do segurado (módulo à parte).

> **Defesa contra prompt injection**
> O texto vindo do OCR é tratado como **dado não confiável**: o envelope de prompt manda ignorar instruções que apareçam dentro do documento e proíbe inventar informações ausentes (campos faltantes ficam vazios).

4.4 Provedores com fallback



Falha = HTTP 5xx, 429 ou timeout de 60 s. Após falhas consecutivas, o circuit breaker abre por 30 s.

Todas as chamadas externas passam pelo **ResilientHttpSender**: timeout por tentativa, retentativas com backoff exponencial (500 ms, 2 s, 6 s) e circuit breaker por host.

5. Pós-processamento clínico e alertas

Depois da extração, regras **determinísticas** (sem IA) corrigem e validam os dados. Isso reduz o risco de a IA errar silenciosamente.

5.1 Normalização de unidades

Laboratórios usam unidades diferentes (SI x ambulatorial). O **UnitNormalizer** converte para uma unidade canônica por analito:

Analito	De	Para	Fator
Glicemia	mmol/L	mg/dL	× 18,02
Colesterol total / HDL / LDL	mmol/L	mg/dL	× 38,67
Triglicerídeos	mmol/L	mg/dL	× 88,57
Creatinina	µmol/L	mg/dL	× 0,011312
Ácido úrico	µmol/L	mg/dL	× 0,016807
Vitamina D (25-OH)	nmol/L	ng/mL	× 0,4006
Vitamina B12	pmol/L	pg/mL	× 1,355
Hemoglobina	g/L	g/dL	÷ 10

5.2 Validação e confiança

- Extração e validação de **CID-10**, **CNES** e **CRM** contra os catálogos importados.
- Qualquer campo com **confiança menor que 0,80**, ou medicamento sem dosagem legível, marca o documento com `RevisaoPendente`. O usuário vê os campos destacados e só conclui a revisão depois de editar e confirmar cada um.

5.3 Alertas clínicos (AlertaMedicamentoService)

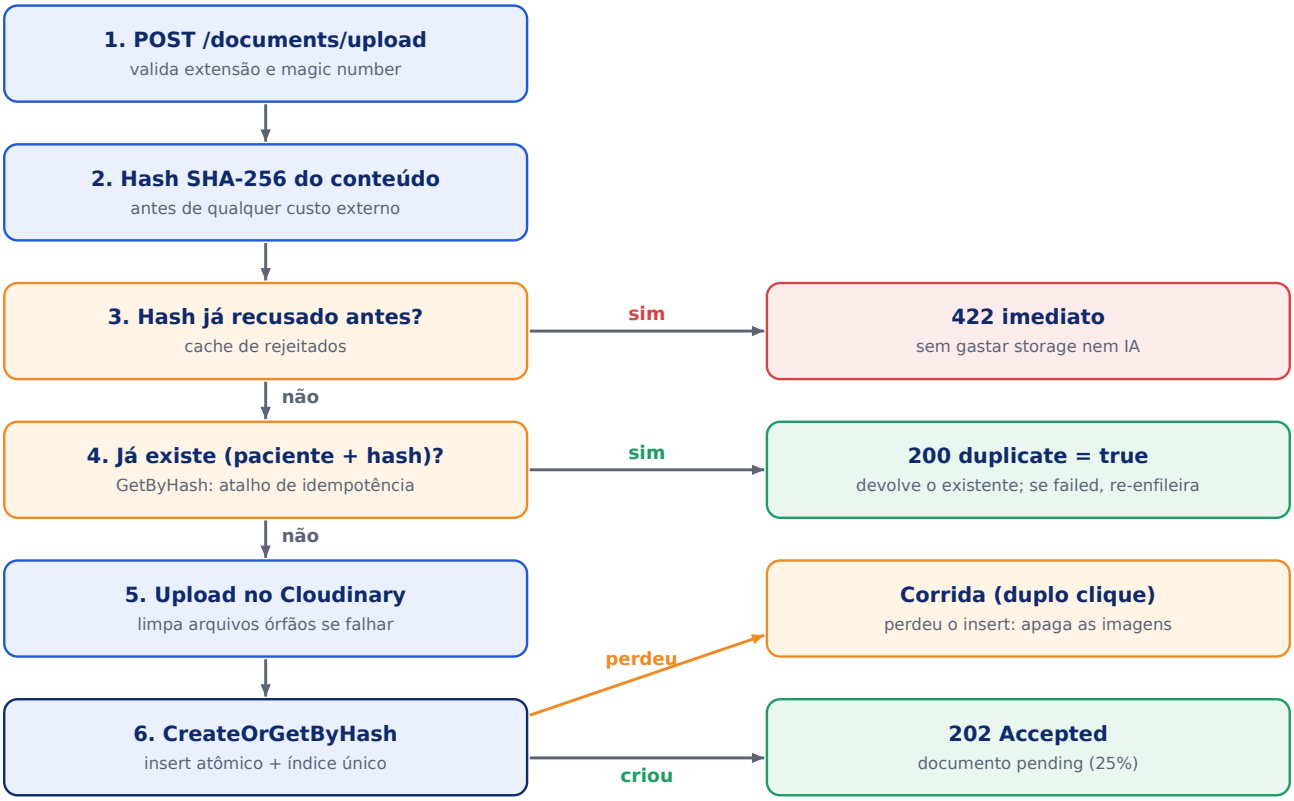
Camada	Tipo	Como funciona
1	Duplicidade (determinística)	Compara os medicamentos novos entre si e com os medicamentos contínuos da ficha médica.
2	Alergia (determinística)	Cruza com as alergias da ficha, expandidas por classe farmacológica (ex.: dipirona ↔ metamizol; penicilina ↔ amoxicilina).
3	Interação fármaco-fármaco (IA)	O Gemini (temp 0) recebe a lista de substâncias e devolve interações com severidade (baixa, moderada, alta), explicação e recomendação de conversar com o médico ou farmacêutico.

Cada alerta tem tipo, severidade, mensagem, medicamentos envolvidos e pode ser **dispensado** pelo usuário. Alertas que dependem de IA são marcados como retentáveis se o provedor falhar.

6. Idempotência

Sem cuidado, um duplo clique, uma queda de rede ou o reenvio do mesmo arquivo gastaria OCR e tokens de IA de novo e geraria documentos duplicados. O sistema trata isso em **três níveis**.

6.1 Upload: mesmo arquivo = mesmo documento



- **Hash SHA-256** do conteúdo do upload, calculado antes de qualquer custo externo.
- **Índice único parcial** `ux_pacienteId_fileHash` no MongoDB: o banco, e não só o código, impede duplicatas. É parcial para não quebrar documentos legados sem hash.
- **CreateOrGetByHash**: se duas requisições concorrentes chegam juntas, uma cria e a outra recebe o documento existente; a perdedora apaga as imagens que acabou de subir.
- **Reenvio de falha**: se o documento existente está em *failed*, reenviar o mesmo arquivo o recoloca na fila de forma atômica (retorna falso se já foi re-enfileirado).
- **Cache de rejeitados**: um arquivo já recusado como inválido é recusado de novo na hora, sem gastar storage, OCR ou IA.

6.2 Fila: processamento exclusivo

O lock atômico por *FindOneAndUpdate* garante que um documento seja processado por um worker de cada vez, e o lock expira para recuperar falhas.

6.3 Indexação vetorial: reprocessar não duplica

O `UpsertChunksAsync` apaga todos os chunks anteriores do documento antes de gravar os novos. O ID de cada chunk é determinístico (`{documentoId}_c{índice}`). Reprocessar ou reindexar (`POST /api/chat/reindex`) nunca polui o espaço vetorial. Há teste automatizado para isso (DocumentVectorIndexer_IdempotentlyIndexesDocument).

6.4 Migrações

As migrações de banco são versionadas, registradas (MigracaoAplicada) e idempotentes, e rodam antes de a aplicação aceitar tráfego.

7. Chat com RAG híbrido

O chat responde com base nos documentos reais do paciente, sem expor o banco inteiro ao modelo e sem permitir que a IA invente fatos.

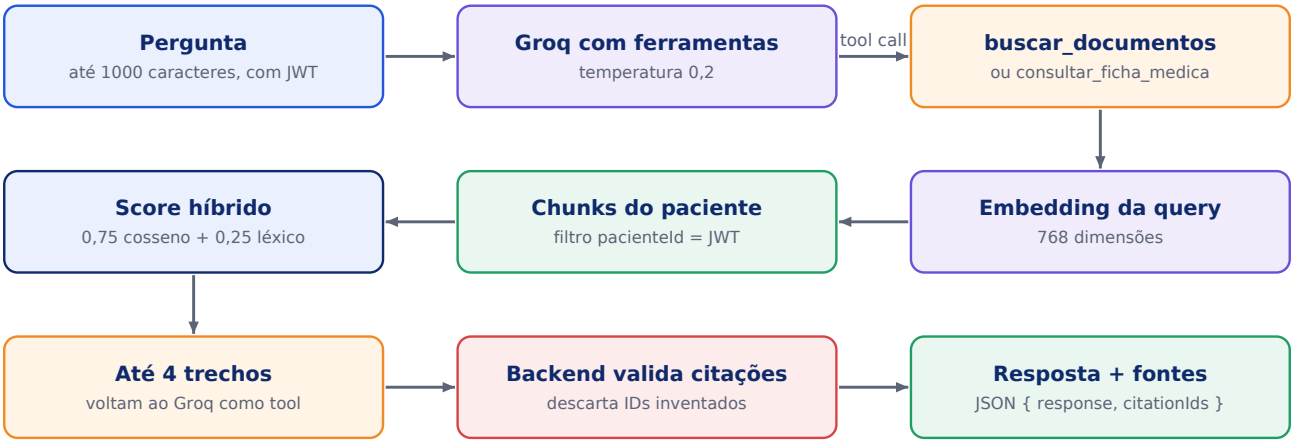
7.1 Indexação (chunking clínico)

Quando um documento fica pronto, o **DocumentChunker** o divide por significado, não por contagem cega de caracteres:

Chunk	Conteúdo	Serve para perguntas como
Cabeçalho e síntese	Título, tipo, data, médico, clínica, especialidade, resumo, diagnóstico e conclusões.	“Qual foi o diagnóstico da última consulta?”
Itens estruturados	Medicamentos (nome, dosagem, horário) e resultados de exame (valor, unidade, referência).	“Qual minha glicose no último exame?”
Narrativa	Texto de laudos e observações em blocos de até 800 caracteres, respeitando pontuação.	Detalhes de laudos e orientações.

Cada chunk vira um vetor de **768 dimensões** (embeddings do Gemini, em lote, timeout de 15 s) e é guardado no próprio MongoDB (coleção `DocumentoChunks`), com índices por paciente/documento e por paciente/tipo. Não há banco vetorial externo: a similaridade é calculada no runtime .NET.

7.2 Consulta



Se os embeddings falharem ou não houver vetores, a busca lexical assume sem derrubar o chat.

Score híbrido = 0,75 × similaridade de cosseno + 0,25 × sobreposição lexical. A parte semântica entende que “remédio para pressão alta” combina com “Losartana”; a parte lexical acerta dosagens e nomes comerciais raros. Entram só chunks com score > 0,25 (ou cosseno > 0,35); é escolhido o melhor chunk por documento, até **4 documentos** de até 1.500 caracteres.

7.3 Garantias do chat

- **Isolamento por paciente**: o identificador vem do token JWT autenticado. As ferramentas do modelo não têm parâmetro de paciente.

- **Somente leitura**: duas ferramentas, buscar_documentos_relevantes (tipo e limite opcionais) e consultar_ficha_medica (sem CPF, RG ou endereço).
- **Antialucinação**: a resposta é um JSON `{ response, citationIds }`; o backend descarta qualquer ID que não esteja entre os documentos realmente recuperados naquele turno.
- **Orçamento de contexto**: entrada de 12.000 tokens (padrão) e saída de 800; o histórico (até 6 trocas) é podado em pares até caber.
- **Consentimento** de chat obrigatório, limite de 1.000 caracteres por pergunta e 10 requisições por minuto.
- **Degradação graciosa**: se os embeddings caírem ou ainda não houver vetores, a busca lexical assume.

8. Segurança, privacidade e LGPD

8.1 Autenticação e proteção de requisições

- **JWT** em cookie `HttpOnly`, com endpoints de login, logout, refresh, alteração e recuperação de senha (e-mail via Brevo).
- **Senhas com BCrypt**. No login, quando o e-mail não existe, o sistema faz mesmo assim uma verificação BCrypt contra um hash fictício, para o tempo de resposta não revelar quais contas existem.
- **Proteção CSRF**: toda chamada mutável (POST, PUT, PATCH, DELETE) exige o header `X-Requested-With: XMLHttpRequest`; o cliente web o envia sozinho.
- **CORS** por allowlist de origens, **cabeçalhos de proxy** tratados (X-Forwarded-*) e **tratamento global de exceções**.
- **Validação de upload**: extensões permitidas (pdf, jpg, jpeg, png), conferência dos *magic numbers* do arquivo, até 10 MB por arquivo.

8.2 Limites de requisições (rate limiting)

Política	Limite	Política	Limite
register	5 / min	upload	20 / min
login	10 / min	chat	10 / min
refresh	20 / min	emergencia	10 / min
recuperar senha	3 / 10 min	redefinir senha	5 / 10 min

8.3 Link de emergência

Pensado para socorro: um token gera um link/QR que mostra **só o essencial** (nome, idade, sexo, tipo sanguíneo, alergias, doenças crônicas, medicamentos contínuos, contato de emergência, doador de órgãos e cirurgias).

- Token aleatório de 32 bytes; no banco fica **apenas o hash SHA-256**, e o token nunca vai para log.
- **Validade curta**: 12 horas por padrão, máximo de 7 dias; pode ser rotacionado ou **revogado** a qualquer momento (invalida links e QR já compartilhados).
- Resposta com `Cache-Control: no-store` e `Referrer-Policy: no-referrer`; contas em exclusão não respondem.
- Cada acesso anônimo gera registro de **auditoria**.

8.4 Auditoria, consentimento e exclusão

- **Auditoria de acesso**: registra quem acessou o quê, quando, o resultado e o IP **truncado**, mas nunca conteúdo clínico.
- **Consentimentos separados** para OCR/IA e para chat; revogar bloqueia imediatamente as rotas de IA.
- **Sem PHI em logs e métricas**: apenas duração, contagem de tokens e IDs de correlação.
- **Relatório clínico desidentificado**: antes de chamar o modelo, a data de nascimento vira idade e CPF, e-mail e telefone são omitidos.
- **Exclusão de conta (LGPD, art. 18)**: o AccountDeletionService apaga em cascata registros, imagens no Cloudinary, histórico de chat, auditoria e **todos os vetores**.

9. Insights, catálogos e demais módulos

9.1 Série histórica de exames

A tela de Insights monta gráficos por analito (glicemia, colesterol, TSH, creatinina, vitamina D etc.) a partir dos resultados estruturados de cada exame, com **faixas de referência** e status. O backend aplica **limites fisiológicos plausíveis** por analito e marca como *outlier* valores impossíveis (por exemplo, um erro de leitura do OCR), em vez de distorcer o gráfico.

9.2 Relatório clínico consolidado

Gera um resumo cronológico para levar à consulta: janela de 1 a 120 meses (padrão 6), até 50 documentos prontos e a ficha médica, usando o Gemini com temperatura 0,2, limite de 1.200 tokens de saída e orçamento de entrada de 12.000 tokens. O paciente é desidentificado antes da chamada.

9.3 Leitor de carteirinha

O usuário fotografa a carteirinha do plano; o mesmo OCR duplo e a unificação rodam e um prompt específico extrai operadora e número, preenchendo o cadastro.

9.4 Catálogos de referência

Catálogo	Uso
CID-10	Validar e descrever diagnósticos citados em atestados e laudos; consulta na tela de Insights.
CNES	Validar o estabelecimento de saúde citado no documento.
Medicamentos (ANVISA)	Consultar descrição e bula de medicamentos; apoiar o dicionário clínico usado nos alertas.

9.5 Telas do frontend

Início, Recursos, Sobre, Autenticação e Redefinição de senha; Dashboard, Upload, Documentos e Detalhe do documento, Prontuário, Anamnese, Chat, Insights, Perfil, Emergência; Privacidade e Termos. A interface é responsiva e tem tema escuro.

10. Resiliência e observabilidade

10.1 ResilientHttpSender

- **Timeouts**: OCR 45 s, extração com IA 60 s, chat 25 s, embeddings 15 s.
- **Retentativas** com backoff exponencial para 429 e 5xx (500 ms, 2 s, 6 s).
- **Circuit breaker em memória**: após falhas consecutivas o circuito abre por 30 s e rejeita chamadas na hora, evitando esgotar threads.

10.2 Métricas e custo

O meter `SaudeMemora.AI` emite **ai_requests_total** (por operação, provedor e status), **ai_duration_ms**, **ai_tokens_consumed** e **ai_estimated_cost_usd**, calculado pelo AiCostEstimator a partir dos preços por milhão de tokens configurados. Isso permite saber quanto custa cada documento processado e cada resposta de chat.

10.3 Testes

O backend tem 16 arquivos de teste, cobrindo autenticação, CSRF, CORS, consentimento, token de emergência, exclusão de conta, concorrência do worker, RAG e indexação vetorial idempotente, chat com IA, catálogos, normalização de faixas de referência e o ResilientHttpSender. O frontend e os scripts têm testes para busca de documentos, erros de upload e consulta de bulas.

11. Stack, configuração e implantação

11.1 Tecnologias

Área	Tecnologia
Frontend	React + Vite, TailwindCSS, React Query, Wouter, Recharts, tema claro/escuro.
Backend	C#, ASP.NET Core 9 (Minimal API), BackgroundService para a fila.
Banco e cache	MongoDB (dados e vetores); Redis opcional para cache.
Armazenamento	Cloudinary (imagens enviadas).
IA	OCR.Space (leitura), Gemini (extração, unificação, interações, relatório e embeddings), Groq (chat e fallback de extração).
E-mail	Brevo (recuperação de senha).
Implantação	Vercel (frontend + API em container); docker-compose para desenvolvimento com API, MongoDB e Redis.

11.2 Variáveis de ambiente principais

Variável	Padrão	Função
GEMINI_API_KEY / GEMINI_MODEL	obrigatória / configurável	Extração, unificação, interações e relatório.
GEMINI_EMBEDDING_MODEL	text-embedding-004	Vetores de 768 dimensões.
GROQ_API_KEY / GROQ_MODEL	obrigatória / configurável	Chat com tool calling e fallback de extração.
GROQ_CHAT_TOOL_CALLING_ENABLED	true	Liga as ferramentas do chat.
GROQ_CHAT_INPUT_TOKEN_BUDGET	12000	Orçamento de entrada (4.096 a 64.000).
GROQ_CHAT_OUTPUT_TOKEN_LIMIT	800	Máximo de tokens da resposta (128 a 4.096).
GEMINI_REPORT_INPUT_TOKEN_BUDGET	12000	Orçamento de entrada do relatório.
OCR_SPACE_API_KEY	obrigatória	Acesso aos motores de OCR.
DOCUMENT_WORKER_CONCURRENCY	3	Documentos em paralelo por instância (1 a 16).
EMERGENCIA_TOKEN_HORAS_EXPIRACAO	12	Validade do token de emergência (máx. 168 h).
JWT_SECRET_KEY, MONGODB_*, CLOUDINARY_*	obrigatórias	Autenticação, banco e storage.

11.3 Principais endpoints

Grupo	Rotas
Autenticação	POST /api/auth/register, login, logout, refresh, alterar-senha, forgot-password, reset-password
Documentos	GET /api/documents, /count, /{id}; POST /upload, /{id}/retry, /{id}/reprocessar; PUT e DELETE /{id}
Revisão e alertas	POST /api/documents/{id}/revisao/confirmar e /alertas/{alertaId}/dispensar
Chat	POST /api/chat; GET /api/chat/history, /history/days, /documentos-consultados; POST /api/chat/reindex
Paciente	GET/DELETE /api/pacientes/me; PATCH /perfil e /contato; /carteirinha; /consentimento
Ficha médica	GET e PATCH /api/ficha-medica/me
Emergência	POST /api/pacientes/me/emergencia (e /rotate); DELETE; GET /status; GET público /api/emergencia/{token}
Insights	GET /api/dashboard/resumo; /api/exames (analitos, série); GET /api/reports/generate; /api/catalogo-cid10, -cnes, -medicamentos
Saúde da API	GET /healthz, /api/ping

12. Decisões de projeto em resumo

Decisão	Motivo
Processamento assíncrono com fila no MongoDB	Upload responde rápido; o trabalho pesado não bloqueia a requisição e se recupera de falhas por lock com expiração.
Idempotência em três níveis	Evita custo de IA, duplicatas e corridas, com garantia no próprio banco.
OCR duplo + unificação sem tocar em números	Cobre texto corrido e tabelas sem arriscar dosagens.
Regras determinísticas ao redor da IA	Conversão de unidades, validação de catálogos e alertas básicos não dependem do modelo.
Revisão humana por confiança	Campos incertos são mostrados ao usuário em vez de aceitos em silêncio.
Vetores no próprio MongoDB	Menos infraestrutura e custo; isolamento por paciente direto na consulta.
Paciente sempre vindo do JWT	O modelo não consegue acessar dados de outra pessoa, mesmo manipulado.
Citações validadas no backend	O usuário só vê fontes que de fato embasaram a resposta.
Privacidade por padrão	Consentimento, auditoria sem conteúdo clínico, IP truncado, token de emergência com hash e expiração, exclusão em cascata.

> **Observação sobre este documento**
> O conteúdo foi levantado a partir do código-fonte e da documentação do repositório. Parâmetros como limites, timeouts e pesos refletem o que está documentado e configurado no projeto e podem mudar nas próximas versões.